Python cho Khoa học dữ liệu

Bài 2 — Cú pháp cơ bản: một vòng Python



Hà Minh Tuấn
hmtuan@hcmus.edu.vn

Khoa Toán - Tin học
Trường Đại học Khoa học Tự nhiên
ĐHQG, HCM

September 21, 2026

Nội dung bài học

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

Roadmap

1 Chạy trước, hiểu sau
2 Làm việc với Python
3 Biến, kiểu dữ liệu, toán tử
4 Chuỗi ký tự
5 Cấu trúc điều khiển
6 Cấu trúc dữ liệu
7 Hàm
8 Nhập, xuất, lỗi và thư viện
9 Tổng kết

Bài này học theo cách khác

Bài 1 lo phần công cụ, và ta chưa viết dòng Python nào. Bài này mở đầu bằng một chương trình **hoàn chỉnh**: đọc tệp điểm, tính toán, in báo cáo. Sau đó mới tách từng mảnh ra học.

Thu muc mot_vong_python

```
mot_vong_python/
|-- diem.txt                 # du lieu: 8 sinh vien
|-- mot_vong_python.py       # ca chuong trinh trong mot tep
`-- mot_vong_python.ipynb    # cung chuong trinh, chia thanh tung o
```

Học xong bài này, bạn

chạy được chương trình, **chỉ ra được mỗi dòng thuộc nhóm kiến thức nào**, tự viết được những đoạn ngắn với số, chuỗi, `if`, `for`, list, dict, hàm, và **đọc được thông báo lỗi**. Từ Bài 3 đến Bài 7 ta quay lại từng nhóm và đi sâu.

Dữ liệu, và cách chạy

diem.txt — moi dong: ten;lop;diem

```
Nguyen Van An;24TTH;8.5
Tran Thi Binh;24TTH;4.0
Le Van Chi;25KDL;9.0
Pham Thu Dung;25KDL;6.5
Hoang Minh Giang;24TTH;5.0
Vo Thi Ha;25KDL;3.5
Do Quoc Khanh;24TTH;7.5
Bui Lan Anh;25KDL;10.0
```

```
python mot_vong_python.py
```

Trước đó, mở thư mục `mot_vong_python` bằng **File** → **Open Folder**. Gặp `FileNotFoundError: ... 'diem.txt'` nghĩa là terminal đang đứng sai thư mục.

Kết quả

Nhan Enter khi chuong trinh hoi nguong

```
Nhap nguong diem dat (Enter = 5.0):

===== BAO CAO DIEM =====
So sinh vien : 8
Cac lop      : ['24TTH', '25KDL']
Nguong dat   : 5.0
Trung binh   : 6.75
Ti le dat    : 75.0%
...
Diem thap nhat can cong 0.5 diem 3 lan thi dat.
```

Phần `...` là bảng xếp loại và trung bình từng lớp, sẽ gặp ở các slide sau. Thử gõ `6.5` thay vì Enter: tỉ lệ đạt thành 62.5%, dòng cuối thành `6 lan thi dat.`

Năm nhóm kiến thức nằm ở đâu trong chương trình

Nhóm	Trong chương trình	Học kỹ ở
1. Biến, kiểu, toán tử	`so_sinh_vien`, `nguong_dat`, `dat`, `so_dat / so_sinh_vien * 100`	Bài 3
2. `if`/`else`, `for`, `while`	xếp loại, đếm số người đạt, vòng `while` ở cuối	Bài 4
3. `list`, `tuple`, `dict`, `set`	`ban_ghi` và các tuple trong đó, `cac_lop`, `diem_theo_lop`	Bài 6
4. Hàm	`xep_loai`, `trung_binh`	Bài 5
5. Nhập, xuất	`input()`, `print()`, `open("diem.txt")`	Bài 3, 7

Tệp `mot_vong_python.py` có đánh dấu `# Nhóm 1` … `# Nhóm 5` trước mỗi đoạn. Mỗi phần sau đây mở đầu bằng những mảnh nhỏ tự chạy được, rồi quay về đúng đoạn tương ứng trong chương trình. Chuỗi ký tự (`str`) có hẳn một phần riêng, vì dữ liệu đọc từ tệp luôn là chuỗi.

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

Hai cách chạy Python: tương tác và tệp .py

Gõ `python` trong terminal: dấu nhắc `>>>` hiện ra. Gõ một dòng, Enter, thấy kết quả ngay.

Che do tuong tac

```
>>> 7 + 2
9
>>> ten = "An"
>>> ten
'An'
>>> len(ten)
2
>>> ten * 3
'AnAnAn'
```

Gõ `exit()` để thoát.

- Ở chế độ tương tác, gõ tên một biến là thấy giá trị — không cần `print`. Chuỗi hiện kèm dấu nháy: `'An'`.
- Chạy một tệp (`python ten_tep.py`) thì **chỉ những gì được `print` mới hiện ra.**
- Ô notebook ở giữa hai cách: nó tự hiện giá trị của **dòng cuối** trong ô.

Thói quen tốt

Thử một ý nhỏ ở `>>>` hoặc một ô notebook, rồi mới chép vào tệp `.py`.

print(): in nhiều giá trị, sep và end

```
print("Xin chao Python!")
print("Diem:", 8.5, "- Lop:", "24TTH")
print("a", "b", "c", sep="-")
print("Dong nay khong xuong dong", end=" | ")
print("vi end=' | '")
```

Kết quả

```
Xin chao Python!
Diem: 8.5 - Lop: 24TTH
a-b-c
Dong nay khong xuong dong | vi end=' | '
```

- Truyền nhiều giá trị cách nhau bởi dấu phẩy: `print` tự chèn một dấu cách ở giữa, và số không cần đổi sang chuỗi.
- `sep=` đổi thứ chèn ở giữa; `end=` đổi thứ in ở cuối (mặc định là xuống dòng).

Chú thích và thụt lề

```
# Dong bat dau bang dau # la chu thich, Python bo qua
diem = 8.5           # chu thich o cuoi dong
if diem >= 5:
    print("Dat")     # thut vao 4 dau cach: thuoc khoi if
print("Xong")        # khong thut: nam ngoai khoi if
```

Kết quả

```
Dat
Xong
```

Quen thut le

```
diem = 8.5
if diem >= 5:
print("Dat")
```

Đoạn ở giữa không chạy được:
`IndentationError: expected an indented block after 'if' statement on line 2`

Dòng kết thúc bằng dấu : mở ra một khối; mọi dòng trong khối thụt vào **4 dấu cách**. VS Code tự thụt khi bạn Enter sau dấu :.

Đọc một thông báo lỗi — đọc từ dưới lên

Tệp `tinh_tb.py` có ba dòng: tạo list điểm, tính tổng, in trung bình. Chạy nó:

Chạy: python tinh_tb.py

```
Traceback (most recent call last):
  File "C:\...\tinh_tb.py", line 3, in <module>
    print("Trung binh: " + tong / len(diem))
          ~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~
TypeError: can only concatenate str (not "float") to str
```

1. **Dòng cuối**: loại lỗi (`TypeError`) và lời giải thích: chỉ nối được `str` với `str`, ở đây lại có một `float`.
2. **Phía trên**: tệp nào, `line 3`, và dấu `^` chỉ đúng phép toán gây lỗi.
3. Sửa: `"Trung binh: " + str(tong / len(diem))`, hoặc gọn hơn `print("Trung binh:", ...)`.

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

Phép gán

```
x = 5
x = x + 1          # ve phai tinh truoc: x thanh 6
dem = 0
dem += 1           # viet tat cua: dem = dem + 1
a, b = 1, 2        # gan nhieu bien mot luc
a, b = b, a        # hoan doi, khong can bien tam
print(x, dem, a, b)
```

Kết quả

```
6 1 2 1
```

- = nghĩa là "tính vế phải, rồi gắn tên ở vế trái vào kết quả" — không phải dấu bằng trong toán. Vì vậy `x = x + 1` hoàn toàn hợp lệ.
- +=, −=, *=, /= là cách viết tắt; `dem += 1` gặp ở mọi vòng lặp đếm.

Đặt tên biến

Luật (sai là không chạy):

- Chỉ gồm chữ, chữ số, dấu _; không bắt đầu bằng chữ số.

  `2lop = ...` → `SyntaxError: invalid decimal literal`

- Không trùng 35 từ khoá (`if`, `for`, `class`, `def`, …).

  `class = ...` → `SyntaxError: invalid syntax`

- Phân biệt chữ hoa, chữ thường: `diem` và `Diem` là hai biến khác nhau.

Quy ước (nên theo):

- Chữ thường, nối bằng _: `so_sinh_vien`, `nguong_dat`.
- Tên nói lên ý nghĩa: `diem_tb` thay vì `x`, `d2`.
- Không dùng tên hàm có sẵn làm tên biến: `list`, `sum`, `max`, `input`… Đặt `sum = 10` xong thì hàm `sum()` không gọi được nữa.
- Tên có dấu (`điểm = 8`) vẫn chạy, nhưng đừng dùng: khó gõ, dễ lỗi khi chia sẻ mã.

Bốn kiểu dữ liệu cơ bản

O dau tien trong notebook

```
ten = "Nguyen Van An"      # str   - chuỗi ký tự, đặt trong dấu nháy
so_bai_nop = 3             # int   - số nguyên
diem = 8.5                 # float - số thực, có dấu chấm thập phân
da_nop_du = True           # bool  - chỉ có True hoặc False

print(type(ten), type(so_bai_nop), type(diem), type(da_nop_du))
```

Kết quả

```
<class 'str'> <class 'int'> <class 'float'> <class
↪  'bool'>
```

- Biến là một **cái tên** gắn với một giá trị. Không cần khai báo kiểu trước.
- `type()` cho biết kiểu. Có dấu chấm thập phân thì là `float`; trong nháy thì là `str`, kể cả `"8.5"`.

Số nguyên, số thực, và kiểu đi theo giá trị

```
print(2 ** 100)              # int: lon bao nhieu cung duoc
print(0.1 + 0.2)             # float: co sai so rat nho
print(round(0.1 + 0.2, 2))   # lam tron khi can in ra
x = 42
print(type(x))
x = "bon muoi hai"           # cung ten x, nay giu mot chuoi
print(type(x))
```

Kết quả

```
1267650600228229401496703205376
0.30000000000000004
0.3
<class 'int'>
<class 'str'>
```

- `int` không bị tràn: 2^{100} ra đủ 31 chữ số.
- `float` lưu ở hệ nhị phân nên `0.1 + 0.2` lệch một chút — mọi ngôn ngữ đều vậy. Đừng so sánh hai số thực bằng `==`.
- Kiểu đi theo **giá trị**, không theo tên biến.

Toán tử số học và thứ tự tính

```
print(7 + 2, 7 - 2, 7 * 2, 7 / 2)   # cộng, trừ, nhân, chia
print(7 // 2, 7 % 2, 2 ** 10)       # chia nguyên, lấy dư, lũy thừa
print(8 / 4)                        # chia hết vẫn ra số thực
```

Kết quả

```
9 5 14 3.5
3 1 1024
2.0
```

```
print(2 + 3 * 4)        # nhan truoc cong
print((2 + 3) * 4)      # trong ngoac truoc
print(10 - 4 - 3)       # cung cap: tinh tu trai sang
print(2 ** 3 ** 2)      # rieng **: tinh tu phai sang
```

Kết quả

```
14
20
3
512
```

/ luôn cho số thực, kể cả khi chia hết; // lấy phần nguyên, % lấy phần dư. Thứ tự: **, rồi * / // %, rồi + −. Không chắc thì thêm ngoặc.

Toán tử so sánh và toán tử logic

Voi diem = 8.5 tu o dau tien

```
print(diem >= 5, diem == 10, diem != 10)
print(diem >= 5 and diem < 8)
```

Kết quả

```
True False True
False
```

- So sánh cho ra `bool`: >=, <=, >, <, == (bằng), != (khác).
- Ghép điều kiện bằng `and`, `or`, `not`. Dòng thứ hai cho `False` vì 8.5 không nhỏ hơn 8.
- = là gán, == là so sánh — hai thứ khác hẳn nhau.

Đổi kiểu (ép kiểu)

```
print(int("42") + 1)        # chuoi -> so nguyen
print(float("8.5") * 2)     # chuoi -> so thuc
print(str(42) + "!")        # so -> chuoi
print(int(3.9))             # cat bo phan le, KHONG lam tron
print(bool(0), bool(""), bool("0"))
```

Kết quả

```
43
17.0
42!
3
False False True
```

- Tên kiểu dùng như một hàm: `int()`, `float()`, `str()`, `bool()`.
- `bool` của số 0 và chuỗi rỗng là `False`; chuỗi `"0"` không rỗng nên là `True`.
- Chuỗi không đúng dạng thì báo lỗi: `int("8.5")` → `ValueError: invalid literal for int() with base 10: '8.5'`. Muốn thì đi hai bước: `int(float("8.5"))`.

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

Viết một chuỗi

```
a = 'nhay don'
b = "nhay kep, chua duoc 'nhay don' ben trong"
c = """ba nhay:
viet duoc tren nhieu dong"""
print(b)
print(c)
print("Dong 1\nDong 2\tsau mot tab")
```

Kết quả

```
nhay kep, chua duoc 'nhay don' ben trong
ba nhay:
viet duoc tren nhieu dong
Dong 1
Dong 2  sau mot tab
```

- Nháy đơn hay kép đều được; chọn loại không có trong chuỗi.
- Ba nháy: chuỗi nhiều dòng (và docstring).
- \n xuống dòng, \t tab.

Nối, lặp, đo độ dài

```
ho, ten = "Nguyen", "An"
print(ho + " " + ten)        # noi chuoi
print("=" * 20)              # lap chuoi
print(len("Python"))         # do dai
print("Diem: " + str(8.5))   # so phai doi sang chuoi moi noi duoc
```

Kết quả

```
Nguyen An
====================
6
Diem: 8.5
```

Cạm bẫy thường gặp

`"Diem: " + 8.5` báo `TypeError: can only concatenate str (not "float") to str`. Python không tự đổi số thành chuỗi khi dùng `+`; phải `str(8.5)`, hoặc dùng f-string ở slide sau.

Chỉ số và cắt lát

Ký tự	P	y	t	h	o	n
Chỉ số dương	0	1	2	3	4	5
Chỉ số âm	-6	-5	-4	-3	-2	-1

```
s = "Python"
print(s[0], s[1], s[-1], s[-2])
print(s[0:2], s[2:], s[:4], s[::2])
```

Kết quả

```
P y n o
Py thon Pyth Pto
```

- Đếm từ **0**. Chỉ số âm đếm từ cuối: `s[-1]` là ký tự cuối.
- `s[a:b]` lấy từ vị trí a đến **trước** b; bỏ trống a là từ đầu, bỏ trống b là đến hết; `s[::2]` lấy cách một.
- List và tuple cũng đánh chỉ số và cắt lát y hệt.

Phương thức của chuỗi

```
dong = "  Nguyen Van An ; 24TTH  "
print(dong.strip())
print(dong.strip().upper())
print(dong.strip().split(";"))
print("24TTH".replace("TTH", "KDL"))
print("Nguyen Van An".startswith("Ng"), "An" in "Nguyen Van An")
print("Nguyen Van An".find("Van"), "banana".count("a"))
```

Kết quả

```
Nguyen Van An ; 24TTH
NGUYEN VAN AN ; 24TTH
['Nguyen Van An ', ' 24TTH']
24KDL
True True
7 3
```

- Phương thức gọi bằng dấu chấm: `chuoi.ten()`. Gọi nối tiếp được: `dong.strip().split(";")` — đúng như chương trình làm với mỗi dòng của tệp.
- Chuỗi **không sửa được** (bất biến): `s[0] = "J"` báo `TypeError: 'str' object does not support item assignment`. Các phương thức trả về **chuỗi mới**.

f-string: chèn giá trị vào chuỗi

```
ten, diem, si_so = "An", 8.456, 1234
print(f"{ten} duoc {diem} diem")
print(f"{diem:.2f} | {diem:.0f} | {si_so:,} | {0.75:.1%}")
print(f"[{ten:<6}] [{ten:>6}] [{ten:^6}]")
```

Kết quả

```
An duoc 8.456 diem
8.46 | 8 | 1,234 | 75.0%
[An    ] [    An] [  An  ]
```

`:.2f`	2 chữ số thập phân
`:,`	phân cách hàng nghìn
`:.1%`	phần trăm, 1 chữ số thập phân
`:<6 :>6 :^6`	canh trái / phải / giữa, rộng 6 ô

Chữ `f` trước dấu nháy; biểu thức trong `{}` được tính rồi chèn vào. Bảng xếp loại của chương trình dùng `:<18`, `:>5` cho thẳng cột.

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

if / elif / else

```
diem, nop_du = 7.0, True
if diem >= 5 and nop_du:
    print("Qua mon")
elif diem >= 5:
    print("Du diem nhung thieu bai nop")
else:
    print("Chua qua mon")
```

Kết quả

```
Qua mon
```

- Python xét các điều kiện **từ trên xuống**, gặp cái đúng đầu tiên thì chạy khối đó và bỏ qua phần còn lại.
- `elif` và `else` không bắt buộc; có bao nhiêu `elif` cũng được.
- Hàm `xep_loai` trong chương trình là một chuỗi `if/elif/else`.

range() và vòng for

```
print(list(range(5)))          # 0..4
print(list(range(2, 8)))       # tu 2 den 7
print(list(range(0, 20, 5)))   # buoc nhay 5
print(list(range(5, 0, -1)))   # dem nguoc
```

Kết quả

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[0, 5, 10, 15]
[5, 4, 3, 2, 1]
```

```
for ky_tu in "Python":
    print(ky_tu, end=" ")
print()
for i in range(1, 4):
    print(i, "x 7 =", i * 7)
```

Kết quả

```
P y t h o n
1 x 7 = 7
2 x 7 = 14
3 x 7 = 21
```

`range(a, b, buoc)`: các số từ a đến **trước** b (`list(...)` chỉ để nhìn thấy chúng). `for` duyệt được range, chuỗi, list, tuple, dict, tệp.

Mẫu tích luỹ: tính tổng, đếm

```python
tong = 0
for so in range(1, 101):
    tong += so
print("Tong 1..100 =", tong)

dem_chan = 0
for so in range(1, 101):
    if so % 2 == 0:
        dem_chan += 1
print("So chan:", dem_chan)
```

Kết quả

```
Tong 1..100 = 5050
So chan: 50
```

Mẫu dùng nhiều nhất trong xử lý dữ liệu: khởi tạo biến **trước** vòng lặp, cập nhật **trong** vòng lặp, dùng kết quả **sau** vòng lặp.

Lỗi kinh điển: đặt `tong = 0` vào *trong* vòng lặp — tổng bị xoá về 0 mỗi lần.

Trong chương trình: đếm số người đạt

```
so_dat = 0
for diem in cac_diem:
    dat = diem >= nguong_dat        # bool: True hoặc False
    if dat:
        so_dat = so_dat + 1
ti_le_dat = so_dat / so_sinh_vien * 100
```

Kết quả

```
Ti le dat    : 75.0%
```

- Đúng mẫu tích luỹ: `so_dat = 0` trước, `so_dat += 1` bên trong.
- `if dat:` chỉ chạy dòng bên dưới khi `dat` là `True`.

Trong chương trình: in bảng xếp loại

```
for ten, lop, diem in ban_ghi:
    loai = xep_loai(diem, nguong_dat)
    # :<18 là canh trái trong 18 ô, :>5 là canh phải trong 5 ô
    print(f"{ten:<18} {lop:<6} {diem:>5}   {loai}")
```

Kết quả

```
Nguyen Van An      24TTH    8.5   Gioi
Tran Thi Binh      24TTH    4.0   Chua dat
Le Van Chi         25KDL    9.0   Gioi
Pham Thu Dung      25KDL    6.5   Dat
Hoang Minh Giang   24TTH    5.0   Dat
Vo Thi Ha          25KDL    3.5   Chua dat
Do Quoc Khanh      24TTH    7.5   Dat
Bui Lan Anh        25KDL   10.0   Gioi
```

Mỗi phần tử của `ban_ghi` là một bộ ba; `for ten, lop, diem in ...` tách luôn thành ba biến.

while: lặp khi chưa biết trước số lần

```
diem_thap_nhat = min(cac_diem)
so_lan = 0
while diem_thap_nhat < nguong_dat:
    diem_thap_nhat = diem_thap_nhat + 0.5
    so_lan = so_lan + 1
```

Kết quả

```
Diem thap nhat can cong 0.5 diem 3 lan thi dat.
```

- `for` dùng khi biết trước sẽ duyệt qua cái gì. `while` dùng khi chỉ biết **điều kiện dừng**.
- Thân vòng lặp phải có dòng làm điều kiện tiến tới chỗ sai — ở đây là dòng cộng 0.5. Thiếu nó thì vòng lặp chạy mãi; nhấn **Ctrl + C** để dừng.

break và continue

```
n = 0
while True:
    n += 1
    if n % 5 == 0 and n % 7 == 0:
        break           # thoat ngay
print("Nho nhat chia het cho 5 va 7:", n)

for so in range(1, 11):
    if so % 2 == 0:
        continue        # sang lan ke
    print(so, end=" ")
```

Kết quả

```
Nho nhat chia het cho 5 va 7: 35
1 3 5 7 9
```

- `break`: thoát hẳn vòng lặp.
- `continue`: bỏ phần còn lại của lần lặp này, sang lần kế.
- `while True` + `break`: lặp cho tới khi tìm thấy.

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

Bốn cấu trúc dữ liệu — nhìn nhanh

```
diem = [8.5, 4.0, 9.0]                  # list: có thứ tự, sửa được
sv = ("Nguyen Van An", "24TTH", 8.5)    # tuple: có thứ tự, không sửa
so_sv = {"24TTH": 4, "25KDL": 4}        # dict: tra theo khoá
lop = {"24TTH", "25KDL", "24TTH"}       # set: tự bỏ phần tử trùng

print(diem[0], sv[1], so_sv["25KDL"], len(lop))
```

Kết quả

```
8.5 24TTH 4 2
```

	Có thứ tự	Sửa được	Cho trùng	Lấy phần tử
`list [ ]`	có	có	có	theo vị trí `ds[0]`
`tuple ( )`	có	không	có	theo vị trí `t[0]`
`dict {k: v}`	có	có	khoá: không	theo khoá `d["k"]`
`set { }`	không	có	không	chỉ hỏi `x in s`

list: thêm, xoá, lấy phần tử

```
diem = [8.5, 4.0, 6.5]
diem.append(9.0)            # them vao cuoi
diem.insert(0, 7.0)         # chen vao vi tri 0
print(diem, len(diem))
diem.remove(4.0)            # xoa theo gia tri
cuoi = diem.pop()           # lay ra phan tu cuoi
print(diem, cuoi)
print(diem[0], diem[-1], diem[1:3])
```

Kết quả

```
[7.0, 8.5, 4.0, 6.5, 9.0] 5
[7.0, 8.5, 6.5] 9.0
7.0 6.5 [8.5, 6.5]
```

- append, insert, remove, pop **sửa ngay** list gốc.
- Chỉ số và cắt lát giống hệt chuỗi; diem[1:3] là một list mới.

list: tính toán và sửa phần tử

```
diem = [8.5, 4.0, 6.5, 9.0]
print(sum(diem), min(diem), max(diem))
print(sorted(diem), sorted(diem, reverse=True))
print(9.0 in diem, diem.index(6.5))
diem[1] = 5.0                # list SUA DUOC tung phan tu
print(diem)
```

Kết quả

```
28.0 4.0 9.0
[4.0, 6.5, 8.5, 9.0] [9.0, 8.5, 6.5, 4.0]
True 2
[8.5, 5.0, 6.5, 9.0]
```

- `sum`, `min`, `max`, `len`, `sorted` là hàm có sẵn, dùng được cho mọi dãy số. `sorted` trả về list **mới**, list cũ giữ nguyên.
- Truy cập ngoài phạm vi: `[1, 2, 3][3]` → `IndexError: list index out of range`.

tuple: bộ giá trị cố định

```
sv = ("Nguyen Van An", "24TTH", 8.5)
ten, lop, diem = sv          # mo goi: moi phan mot bien
print(ten, "|", lop, "|", diem)
print(sv[0], len(sv))
```

Kết quả

```
Nguyen Van An | 24TTH | 8.5
Nguyen Van An 3
```

- Dùng cho một "bản ghi" có số phần cố định: tên, lớp, điểm. Không sửa được: `sv[2] = 9.0` → `TypeError: 'tuple' object does not support item assignment`.
- **Mở gói** (`ten, lop, diem = sv`) chính là thứ xảy ra trong `for ten, lop, diem in ban_ghi` và trong `a, b = b, a`.

dict: tra cứu theo khoá

```python
sv = {"mssv": "24110001", "ten": "An", "diem": 8.5}
print(sv["ten"], sv.get("lop"), sv.get("lop", "chua co"))
sv["lop"] = "24TTH"          # them khoa moi
sv["diem"] = 9.0             # sua gia tri cu
print(sv)
print("ten" in sv, len(sv))
```

Kết quả

```
An None chua co
{'mssv': '24110001', 'ten': 'An', 'diem': 9.0,
↪  'lop': '24TTH'}
True 4
```

- `d[khoa]` lấy giá trị; khoá không có thì báo `KeyError: 'email'`. `d.get(khoa, mac_dinh)` không báo lỗi mà trả `None` hoặc giá trị mặc định.
- Gán vào khoá chưa có là **thêm**; gán vào khoá đã có là **sửa**.

dict: duyệt, và bài toán đếm tần suất

```python
diem = {"An": 8.5, "Binh": 6.0, "Chi": 9.0}
for ten, d in diem.items():
    print(ten, "->", d)

cau = "an binh an chi an binh"
dem = {}
for tu in cau.split():
    dem[tu] = dem.get(tu, 0) + 1
print(dem)
```

Kết quả

```
An -> 8.5
Binh -> 6.0
Chi -> 9.0
{'an': 3, 'binh': 2, 'chi': 1}
```

- `.items()` cho từng cặp (khoá, giá trị); `.keys()`, `.values()` cho riêng từng phần.
- `dem[tu] = dem.get(tu, 0) + 1`: một dòng đếm mọi thứ — từ, lớp, mức xếp loại.

set: tập hợp không trùng

```
lop = {"24TTH", "25KDL", "24TTH"}   # phan tu trung tu bi bo
print(len(lop), "25KDL" in lop)
lop.add("24KDL")
print(sorted(lop))
a = {"An", "Binh", "Chi"}
b = {"Binh", "Chi", "Dung"}
print(sorted(a & b), sorted(a | b), sorted(a - b))
```

Kết quả

```
2 True
['24KDL', '24TTH', '25KDL']
['Binh', 'Chi'] ['An', 'Binh', 'Chi', 'Dung']
↪  ['An']
```

- & giao, | hợp, – hiệu.
- Set **không có thứ tự**: in thẳng ra thì thứ tự có thể đổi; bọc `sorted()` cho cố định.
- Tập rỗng: `set()`; còn `{}` là dict rỗng.

Trong chương trình: list các tuple

```
ban_ghi = []                          # list rỗng, sẽ chứa các tuple
with open("diem.txt", encoding="utf-8") as tep:
    for dong in tep:                  # mỗi vòng đọc một dòng
        ten, lop, diem_chu = dong.strip().split(";")
        # mỗi sinh viên là một tuple gồm 3 phần tử
        ban_ghi.append((ten, lop, float(diem_chu)))
```

ban_ghi[0], len(ban_ghi)

```
('Nguyen Van An', '24TTH', 8.5)
8
```

- `ban_ghi` là một **list**; mỗi phần tử là một **tuple** ba phần: tên, lớp, điểm.
- Đọc từ tệp ra **luôn là chuỗi**. Phải `float(diem_chu)` mới tính toán được.

Trong chương trình: set bỏ trùng, dict gom theo lớp

```
cac_lop = set()                          # set: tự bỏ phần tử trùng
diem_theo_lop = {}                       # dict: lớp -> list điểm

for ten, lop, diem in ban_ghi:
    cac_diem.append(diem)
    cac_lop.add(lop)
    if lop not in diem_theo_lop:
        diem_theo_lop[lop] = []
    diem_theo_lop[lop].append(diem)
```

Kết quả

```
Lop 24TTH - trung binh 6.25
Lop 25KDL - trung binh 7.25
```

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

Gọi hàm có sẵn

```
print(len("Python"), round(8.456, 1), abs(-3), max([4, 9, 2]))
print(sum([1, 2, 3]), min(5, 2, 8), type(3.14))
```

Kết quả

```
6 8.5 3 9
6 2 <class 'float'>
```

- Gọi hàm: `ten_ham(doi_so, ...)`. Hàm nhận đối số, làm việc, **trả về** một giá trị.
- Đã gặp: `print`, `input`, `type`, `int`, `float`, `str`, `len`, `range`, `sorted`, `sum`, `min`, `max`, `round`, `abs`, `open`.
- Quên cách dùng? `help(round)` ở >>>, hoặc rê chuột lên tên hàm trong VS Code.

Tự định nghĩa hàm: tham số, return, phạm vi

Trong chuong trinh

```
def xep_loai(diem, nguong_dat):
    """Trả về xếp loại của một điểm."""
    if diem >= 8.0:
        ket_qua = "Gioi"
    elif diem >= nguong_dat:
        ket_qua = "Dat"
    else:
        ket_qua = "Chua dat"
    return ket_qua      # ket_qua chỉ tồn tại trong hàm
```

- `diem`, `nguong_dat` là **tham số**; `return` trả kết quả về chỗ gọi: `loai = xep_loai(diem, nguong_dat)`.
- **Phạm vi biến**: `ket_qua` chỉ sống trong hàm. Gõ `print(ket_qua)` ở ngoài thì gặp `NameError: name 'ket_qua' is not defined`.

return khác print

```python
def binh_phuong_in(x):
    print(x * x)                # chi IN ra man hinh

def binh_phuong(x):
    return x * x                # TRA gia tri ve cho cho goi

a = binh_phuong_in(4)
b = binh_phuong(4)
print("a =", a, "| b =", b)
print(binh_phuong(3) + binh_phuong(4))
```

Kết quả

```
16
a = None | b = 16
25
```

- `binh_phuong_in` in 16 ra màn hình nhưng **không trả gì**, nên a nhận None.
- Muốn tính tiếp với kết quả thì phải `return`. Hàm tốt thường `return`; in hay không để chỗ gọi quyết định.

Tham số mặc định, gọi theo tên

```
def chao(ten, loi_chao="Xin chao"):
    return f"{loi_chao}, {ten}!"

print(chao("An"))                              # dung gia tri mac dinh
print(chao("Binh", "Chao buoi sang"))          # truyen theo vi tri
print(chao(loi_chao="Hello", ten="Chi"))       # truyen theo ten
```

Kết quả

```
Xin chao, An!
Chao buoi sang, Binh!
Hello, Chi!
```

- `loi_chao="Xin chao"`: không truyền thì dùng giá trị này.
- Gọi theo tên (`ten="Chi"`) thì thứ tự không quan trọng, và đọc rõ nghĩa hơn. Đã gặp rồi: `print(..., sep="-")`, `sorted(..., reverse=True)`.

Trả về nhiều giá trị

```
def thong_ke(ds):
    return min(ds), max(ds), sum(ds) /
    ↪  len(ds)

thap, cao, tb = thong_ke([8.5, 4.0, 9.0])
print(thap, cao, round(tb, 2))
```

Kết quả

```
4.0 9.0 7.17
```

`return a, b, c` thật ra trả về **một tuple**; phía gọi mở gói thành ba biến.

Trong chuong trinh

```
def trung_binh(cac_so):
    """Trả về trung bình cộng của một danh
    ↪  sách số."""
    tong = 0.0
    for so in cac_so:
        tong = tong + so
    return tong / len(cac_so)
```

Tham số là cả một list. Chương trình gọi `trung_binh` ba lần: cả lớp và từng lớp. Viết một lần, dùng nhiều lần.

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

input() và print() trong chương trình

```python
# input() luôn trả về chuỗi (str), kể cả khi gõ số
tra_loi = input("Nhap nguong diem dat (Enter = 5.0): ")
if tra_loi == "":
    nguong_dat = 5.0
else:
    nguong_dat = float(tra_loi)      # đổi chuỗi thành số thực
```

```python
# :.2f nghĩa là lấy 2 chữ số sau dấu thập phân
print(f"Trung binh   : {trung_binh(cac_diem):.2f}")
```

`input()` **luôn trả về chuỗi**, kể cả khi người dùng gõ số: muốn tính toán phải `float(...)` hay `int(...)`. Gõ chữ thay vì số thì `float()` báo `ValueError` — xem cách bắt lỗi này ở slide sau nữa.

Đọc một tệp văn bản

```
with open("diem.txt", encoding="utf-8") as tep:
    for dong in tep:
        print(repr(dong))
```

Kết quả

```
'Nguyen Van An;24TTH;8.5\n'
'Tran Thi Binh;24TTH;4.0\n'
...
```

- `with open(...) as tep:` mở tệp, và tự đóng tệp khi ra khỏi khối. `encoding="utf-8"`: luôn ghi rõ bảng mã — Bài 7 sẽ giải thích vì sao.
- Mỗi dòng còn dính `\n` ở cuối. `dong.strip().split(";")` bỏ nó đi rồi cắt thành `['Nguyen Van An', '24TTH', '8.5']` — ba *chuỗi*, kể cả điểm.

Ghi ra tệp

```python
with open("ket_qua.txt", "w", encoding="utf-8") as f:
    f.write("Lop 24TTH: 6.25\n")
    f.write("Lop 25KDL: 7.25\n")

with open("ket_qua.txt", encoding="utf-8") as f:
    print(f.read())
```

Kết quả

```
Lop 24TTH: 6.25
Lop 25KDL: 7.25
```

- `"w"`: ghi mới (xoá sạch nội dung cũ nếu tệp đã có); `"a"`: ghi nối vào cuối. Không ghi chế độ thì mặc định là `"r"` — đọc.
- `write` không tự xuống dòng: phải tự thêm `\n`.
- `f.read()` đọc cả tệp thành một chuỗi; `for dong in f` đọc từng dòng.

Bắt lỗi với try / except

```
tra_loi = "tam"                    # nguoi dung go chu, khong go so
try:
    nguong = float(tra_loi)
except ValueError:
    print("Khong phai so, dung nguong mac dinh 5.0")
    nguong = 5.0
print("Nguong dat:", nguong)
```

Kết quả

```
Khong phai so, dung nguong mac dinh 5.0
Nguong dat: 5.0
```

- Đoạn trong `try` gặp lỗi đúng loại ghi ở `except` thì Python nhảy xuống khối `except` thay vì dừng chương trình.
- Ghi rõ loại lỗi (`ValueError`). Viết `except:` trơn sẽ nuốt luôn cả những lỗi bạn cần thấy để sửa.

Dùng thư viện: import

```
import math
import random

print(math.sqrt(16), math.floor(7.8), math.ceil(7.2))
print(math.pi)
random.seed(2026)          # co dinh de lan nao chay cung ra giong nhau
print(random.randint(1, 6), random.choice(["An", "Binh", "Chi"]))
```

Kết quả

```
4.0 7 8
3.141592653589793
1 Binh
```

- math, random có sẵn khi cài Python (thư viện chuẩn). Gọi bằng ten_module.ten_ham.
- Từ Bài 8, ta import theo đúng cách này: import numpy as np, import pandas as pd — chỉ khác là phải pip install trước.

Roadmap

1. Chạy trước, hiểu sau
2. Làm việc với Python
3. Biến, kiểu dữ liệu, toán tử
4. Chuỗi ký tự
5. Cấu trúc điều khiển
6. Cấu trúc dữ liệu
7. Hàm
8. Nhập, xuất, lỗi và thư viện
9. Tổng kết

Tám lỗi bạn sẽ gặp trong tuần đầu

Dòng cuối của thông báo lỗi	Thường là do
`IndentationError: expected an indented block after 'if' statement on line 2`	quên thụt lề sau dấu `:`
`SyntaxError: expected ':'`	quên dấu `:` cuối dòng `if`, `for`, `def`
`NameError: name 'diem_tb' is not defined`	gõ sai tên, hoặc dùng biến trước khi gán
`TypeError: can only concatenate str (not "float") to str`	nối chuỗi với số bằng `+`
`ValueError: invalid literal for int() with base 10: '8.5'`	đổi kiểu một chuỗi không đúng dạng
`ZeroDivisionError: division by zero`	chia cho 0 (vd. lớp không có ai)
`IndexError: list index out of range`	chỉ số vượt quá `len - 1`
`KeyError: 'email'`	khoá không có trong dict

Ghi nhớ

Gặp lỗi: đọc **dòng cuối** trước, rồi xem **số dòng** ở phía trên. Chép nguyên dòng cuối để tra cứu khi chưa hiểu.

Cú pháp của cả bài — một trang

Muốn	Viết
Gán biến, đổi kiểu	`x = 5  x += 1  a, b = b, a  float("8.5")  str(42)`
Tính toán, so sánh	`+ - * / // % **  == != < > <= >=  and or not`
Chuỗi	`s[0] s[-1] s[1:3]  len(s)  .strip() .split(";") .upper()` `f"{x:.2f}"`
Rẽ nhánh	`if ...:  elif ...:  else:`
Lặp	`for x in ds:  for i in range(n):  while dk:  break` `continue`
list	`[1, 2]  .append(x)  .pop()  sum min max sorted`
tuple, dict, set	`(1, 2)  d["k"]  d.get("k", 0)  d.items()  {1, 2}  x in` `s`
Hàm	`def ten(a, b=0):  …  return kq`
Nhập, in	`input("...")  print(a, b, sep=", ")`
Tệp	`with open("t.txt", "w", encoding="utf-8") as f:`
Lỗi, thư viện	`try: … except ValueError:  import math`

Notebook hay tệp .py, và ba việc tự thử

- **Notebook** chạy từng ô, xem kết quả từng bước — dùng để *khám phá*.
- **Tệp** `.py` chạy một lần từ đầu đến cuối — dùng để chứa *chương trình hoàn chỉnh*. Đúng như Bài 1 đã nói.

1. Thêm một dòng vào `diem.txt`, lưu lại, chạy lại chương trình.
2. Trong `xep_loai`, thêm mức `"Xuat sac"` cho điểm từ 9.5 trở lên.
3. In thêm tên của sinh viên có điểm cao nhất.
4. Bọc dòng `float(tra_loi)` bằng `try/except` để gõ chữ không làm chương trình dừng.
5. Đếm mỗi mức xếp loại có bao nhiêu người, dùng một `dict`.

Chuẩn bị cho bài 3

- **Bài 3 — Biến, kiểu dữ liệu và mô hình đối tượng**, tuần sau. Bắt đầu đi sâu vào Nhóm 1: viết `b = a` thì Python thật sự làm gì.
- **Đọc trước:** Ramalho, *Fluent Python* 2nd ed., chương 6.
- **Câu hỏi mang tới lớp:** đoạn mã sau in ra gì, và vì sao?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Câu hỏi?

Hà Minh Tuấn — hmtuan@hcmus.edu.vn